TECHNICAL TRAINING PROGRAM

Training Days 2026 - B

Instructor: Tsounias Ioannis | Technical Training Specialist

SOFTWARE IN IT INFRASTRUCTURE

Software is a fundamental part of every IT infrastructure. While hardware provides the physical resources, software provides the instructions that allow devices, users, and services to perform useful tasks.

Software can be used in both traditional and cloud-based infrastructures. It may run directly on a local computer, on a company server, in a data center, or through a cloud service.

WHAT IS SOFTWARE?

Software is a collection of programs, instructions, and related data that tell a computer or another electronic device what to do.

Unlike hardware, software has no physical form. However, it controls how hardware resources are used and enables users to perform tasks such as:

- Creating documents
- Communicating
- Accessing websites
- Managing business information
- Storing and processing data
- Monitoring and protecting IT systems

MAIN SOFTWARE CATEGORIES

Software can be grouped into three main categories:

1. System Software

System software manages the computer's hardware and provides the environment in which other software can operate.

Examples include:

- Operating Systems
- Device Drivers
- System Utilities

2. Application Software

Application software allows users or organizations to perform specific tasks.

Examples include:

- Web Browsers
- Office Applications
- Communication Tools
- Ticketing Systems
- Customer Relationship Management Systems
- Enterprise Resource Planning Systems

3. Firmware

Firmware is specialized software stored in non-volatile memory inside a hardware device. It provides low-level control and helps the device start, communicate, and perform its intended functions.

Examples include:

- BIOS or UEFI firmware
- Router firmware
- Printer firmware
- Storage-device firmware
- Embedded-device firmware

Firmware is normally developed and maintained by the device manufacturer. Updates must be selected and installed carefully because using incorrect firmware or interrupting the update process may cause the device to malfunction.

OPERATING SYSTEMS

Question: *If a computer had no general-purpose Operating System—no Windows, macOS, or Linux—could a user normally open files, browse the internet, or run everyday applications?*

Answer: In most cases, the answer is no.

The Operating System provides the environment in which users and applications interact with the computer's hardware. Without an Operating System, the device may still execute firmware or basic diagnostic functions, but it cannot normally provide the complete working environment expected by the user.

WHAT IS AN OPERATING SYSTEM?

An Operating System, commonly called an OS, is the main system software that manages a computer's hardware resources and provides services to applications and users.

It acts as an intermediary between:

- The hardware
- Application software
- The user

The OS coordinates the processor, memory, storage, peripheral devices, files, applications, and user accounts.

An Operating System can be compared to the "*conductor of an orchestra*". The conductor does not play every instrument but coordinates them so that they work together correctly.

Without an Operating System, users and applications would need much more direct control of the hardware, making the computer difficult to use and manage.

A LITTLE BIT OF HISTORY

Early computers did not use Operating Systems in the same way that modern computers do. Programs were loaded and executed manually, and a computer normally performed one task at a time.

During the 1950s and 1960s, Operating Systems gradually introduced:

- batch processing
- multiprogramming
- time-sharing
- improved hardware management

These developments allowed computers to run several programs more efficiently and to support interaction with multiple users.

UNIX, developed at the end of the 1960s, became highly influential because of its portability, multitasking capabilities, multi-user design, and modular structure. Many modern Operating Systems have been influenced by UNIX concepts.

Today, Operating Systems are found in:

- desktop computers
- laptops
- servers
- smartphones
- network devices
- vehicles
- industrial systems
- embedded devices

Common examples include *Microsoft Windows, macOS, Linux, Android, and iOS*. Each Operating System is designed for particular users, devices, and operational requirements.

ROLES & FUNCTIONS OF AN OPERATING SYSTEM

An Operating System performs several important functions that allow the computer to operate efficiently, reliably, and securely.

- **Hardware & Device Management**: The OS manages hardware resources and communicates with devices such as:
  - keyboards
  - monitors
  - storage devices
  - printers
  - network adapters
  - other peripherals

It normally uses device drivers to communicate with specific hardware.

- **File & Storage Management**: The OS provides a file system that allows data to be:
  - stored
  - named
  - organized
  - accessed
  - modified
  - deleted

It also manages storage devices, folders, permissions, and available disk space.

- **Process Management**: The OS controls the execution of programs, which are known as processes.
    - starts and stops processes
    - allocates processor time
    - coordinates multitasking
    - helps prevent one process from directly interfering with another
- **Memory Management**:

The OS manages the use of RAM:

    - allocates memory to running processes
    - tracks which memory areas are in use
    - protects the memory used by different processes
    - releases memory when it is no longer required
- **User Interface**:

The OS provides a method through which users can interact with the computer.

The two main types are:

    - **Graphical User Interface (GUI)**: Users interact through windows, icons, menus, and a pointer.
    - **Command-Line Interface (CLI)**: Users enter text commands to perform tasks.
- **User and Access Management**: The OS manages:
    - user accounts
    - authentication
    - groups
    - permissions
    - access to files, applications, devices, and system resources
- **Security and Protection**: The OS provides security mechanisms that help protect the system and its data.

These may include:

    - user authentication
    - access permissions
    - process isolation
    - encryption support
    - firewall functions
    - logging
    - security updates

An Operating System contributes to security, but it must also be configured, updated, and managed correctly.

DIFFERENT TYPES OF OPERATING SYSTEMS

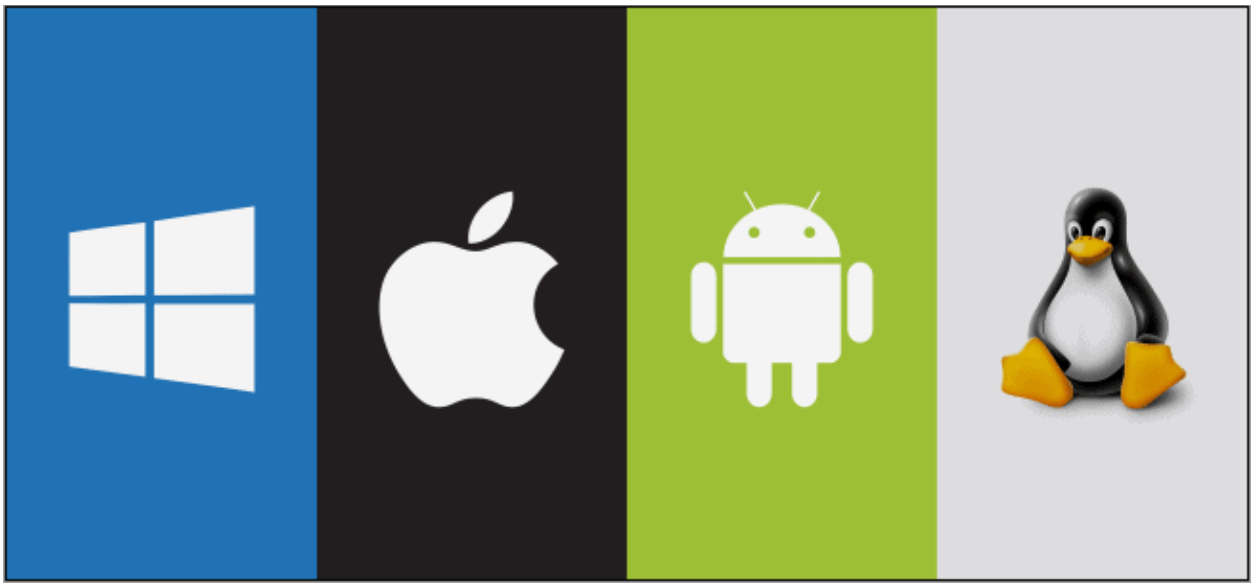
There are different types of Operating Systems. Each type is designed for particular devices, users, and operational requirements.

MAIN TYPES OF OPERATING SYSTEMS

Desktop Operating Systems	Microsoft Windows, macOS, Linux distributions
Server Operating Systems	Windows Server, Linux Server distributions
Mobile Operating Systems	Android, iOS
Embedded Operating Systems	Used in routers, vehicles, industrial equipment, and IoT devices. Examples include FreeRTOS and VxWorks
Real-Time Operating Systems	Designed for systems that must respond within a predictable time, such as industrial control systems, medical devices, and automotive systems.

COMMON OPERATING SYSTEMS AND VENDORS

In this Technical Training Program, we will mainly focus on Desktop and Server Operating Systems. Other types will be introduced briefly to show how Operating Systems are used in different environments.



Different Operating Systems

Microsoft Windows is one of the most widely used Desktop Operating Systems in homes and businesses. It is developed by Microsoft, and the first version of Windows was released in 1985.

Windows is known for its graphical user interface, broad software support, and compatibility with a wide range of hardware and peripheral devices. Users can run business applications, communication tools, creative software, and games.

As already mentioned there are other Operating Systems that were developed or are still being developed by other Vendors.

Apple macOS (Macintosh OS), Apple macOS is the Desktop Operating System developed by Apple for Mac computers. It is one of the main alternatives to Microsoft Windows in the desktop market. MacOS is known for its consistent user interface, close integration with Apple hardware, and compatibility with other Apple services and devices. It is widely used for general productivity, software development, media production, and creative applications.

There is also the well-known Linux, Linux is an open-source, UNIX-like Operating System built around the Linux kernel. It is known for its flexibility, stability, and ability to run on many different types of hardware.

Linux is available through many distributions, also called distros. Examples include Ubuntu, Debian, Fedora, and Red Hat Enterprise Linux. Each distribution combines the Linux kernel with software packages, management tools, and its own support model.

Linux is widely used on servers, cloud platforms, development systems, embedded devices, and desktop computers.

Nowadays, other OS have been developed for other devices like Mobile devices such as:

Android: Android is a mobile Operating System developed primarily by Google and based on the Linux kernel. Its open-source foundation allows device manufacturers to adapt it for different smartphones, tablets, televisions, and other devices.

iOS: iOS is Apple's mobile Operating System for the iPhone. Apple uses a related Operating System called iPadOS for the iPad. It is designed to work closely with Apple hardware and services and supports a wide range of mobile applications.

Other Operating Systems are designed for specialized environments. **UNIX** and **Unix-like** systems are commonly found in enterprise servers, research environments, and technical platforms.

Network devices such as routers and switches also use specialized network Operating Systems. Examples include Cisco IOS, Cisco IOS XE, Junos OS, and ArubaOS.

Many of these systems are managed primarily through a Command-Line Interface, although some also provide graphical or web-based management tools. The CLI gives administrators detailed control, supports automation, and is especially useful for advanced configuration and troubleshooting.

OS PROCESSES & THREADS

A **Process** is a program that is currently running. It includes the program code, the data it uses, and its current execution state. *Each process normally has its own address space in memory*, which helps isolate it from other processes. This improves stability and security because one process cannot normally access or modify another process's memory directly.

A **Thread** is the smallest unit of execution within a process. A process may contain one or more threads. Threads inside the same process *share memory and other resources*, which allows them to work together efficiently. Depending on the system and the number of processor cores, threads may run concurrently or in parallel.

To understand the difference more easily, imagine the computer as a large city.

Each **process** is like a separate building in the city. Each building has its own rooms, functions, and resources. One building might be a school, another a hospital, a factory, or a house. Each building operates separately, although it may still communicate with other buildings when necessary.

Threads can be compared to the people working inside a building. In a hospital, for example, doctors, nurses, and other staff perform different tasks at the same time. They share the hospital's rooms, equipment, electricity, information, and other resources while working towards the same goal.

PROCESS VS THREAD

1. Address Space: Processes normally have separate memory spaces, while threads within the same process share the process's memory and resources. In the city example, buildings have separate internal spaces, while the people inside one building use many of the same facilities and resources.
2. Isolation and Communication: Processes are more strongly isolated from one another. They can still communicate, but they normally require controlled mechanisms provided by the Operating System. Threads inside the same process can communicate and share data more easily because they use the same memory space.
3. Creation & Management: Creating and managing a process normally requires more system resources than creating and managing a thread. In the analogy, constructing and maintaining a separate building requires more effort and resources than adding another worker inside an existing building.

Modern applications often use several processes and threads. For example, a web browser may use separate processes for different tabs or services, while each process may also use multiple threads for tasks such as displaying content, handling user input, and communicating over the network.

APPLICATIONS / APPS

A computer application, also called an app, is software designed to help users perform specific tasks on devices such as computers, tablets, and smartphones. Applications allow us to communicate, organize information, learn, create content, manage business activities, or access entertainment.

Some applications are installed directly on the device, while others run through a web browser or are provided as cloud-based services.

FIRMWARE

Firmware is specialized software stored inside a hardware device. It provides low-level control and allows the device to start, communicate, and perform its intended functions. It provides low-level control for the device's specific hardware. Unlike regular application software, firmware is usually stored in non-volatile memory, such as flash memory, so it remains available when the device is powered off.

BIOS (Basic Input/Output System)

BIOS, stands for *Basic Input/Output System*, is firmware stored on the motherboard. It starts when the computer is powered on, initializes essential hardware, and performs the POST, or Power-On Self-Test.

After these checks, BIOS searches for a bootable device and begins the process of loading the Operating System.

UEFI (Unified Extensible Firmware Interface)

The **UEFI**, which stands for *Unified Extensible Firmware Interface*, is the modern firmware interface used by most current computers. It performs many of the startup and hardware-initialization functions traditionally associated with BIOS, while providing additional capabilities.

Some key points about UEFI:

- **Faster Startup**: UEFI can support faster and more efficient boot processes.
- **Support for Large Storage Devices**: It supports modern partitioning methods, such as GPT, and storage devices larger than those normally supported by traditional BIOS.
- **Graphical Interface**: Some UEFI implementations provide a graphical interface and mouse support.
- **Secure Boot**: Secure Boot helps prevent untrusted boot software from loading during startup.
- **Compatibility Sup**port: Some systems provide a compatibility mode for older Operating Systems or hardware.
- **Network Capabilities**: Certain UEFI implementations support network booting and remote diagnostic functions.

FIRMWARE UPDATES

Firmware updates can improve security, stability, compatibility, and device functionality. However, they should be performed only when required and by following the manufacturer's instructions carefully.

Installing the wrong firmware, interrupting the update, or losing power during the process may cause the device to malfunction or become unusable.

When to Update the Firmware:

- Security vulnerabilities or important security patches
- Known bugs or stability problems
- Compatibility with new hardware or software
- Important manufacturer-recommended improvements
- Required new functionality

How to Perform Firmware Update:

- Identify the exact device model and current firmware version.
- Read the manufacturer's release notes and instructions.
- Confirm that the update is intended for the exact device model and hardware revision.
- Back up important data and configuration settings where possible.
- Download the firmware only from an official and trusted source.
- Ensure that the device has stable electrical power.
- Follow the manufacturer's update procedure without interrupting it.
- Restart the device if required and verify that it operates correctly.

Update:

An update introduces corrections or smaller improvements to existing hardware, software, or services.

Updates may include:

- bug fixes
- security patches
- performance improvements
- compatibility changes
- minor new features

Updates normally keep the same main product or version family.

Upgrade:

An upgrade replaces or moves an existing system, component, or software product to a newer or more capable version.

Examples include:

- replacing a hard disk with an SSD
- increasing the amount of RAM
- replacing an older network device
- or moving from one major software version to another

Upgrades are usually more significant than updates and may require additional planning, compatibility checks, downtime, training, or cost.

Question: *What is the difference between update & upgrade?*

Answer**:** *An update normally improves or corrects an existing version, while an upgrade moves the system or component to a newer or more capable version.*

For example, installing monthly security fixes is an update, while replacing a hard disk with an SSD or moving to a new major Operating System version is an upgrade.

Think about this:

A device is working correctly, but a newer firmware version is available. Should it always be installed immediately? What information should be checked first?